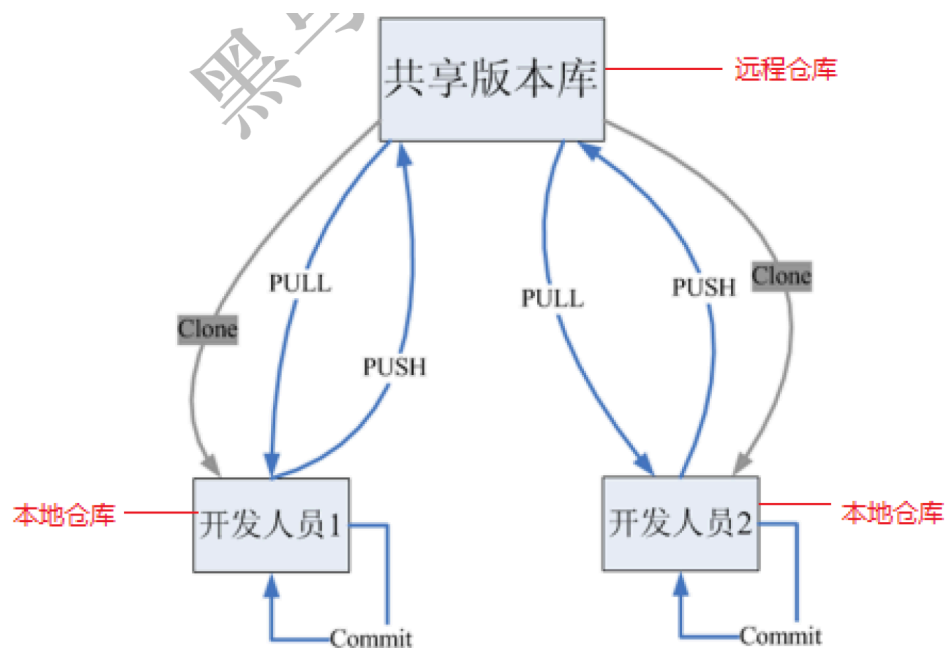


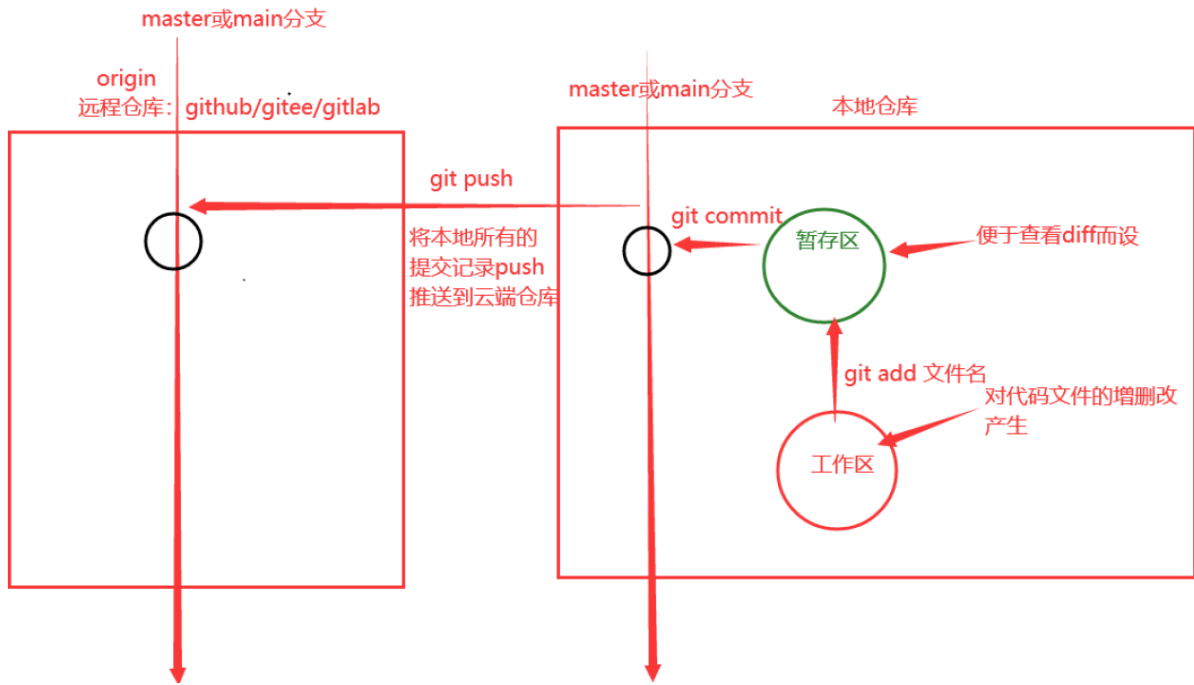
GIT

🕒 Created time	@June 3, 2025 2:08 AM
☰ Tags	

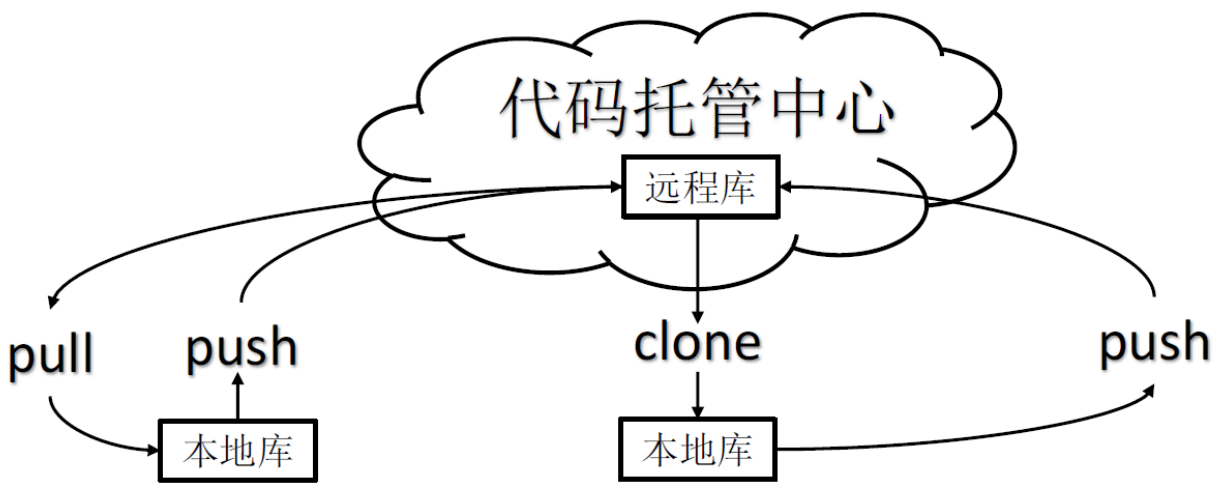
git是分布式 底层是C 不需要有中心服务器 开源分布式版本控制系统 帮助管理linux内核开发而开发的开放源码版本控制软件

gitlab （局域网） /github, gitee （互联网）

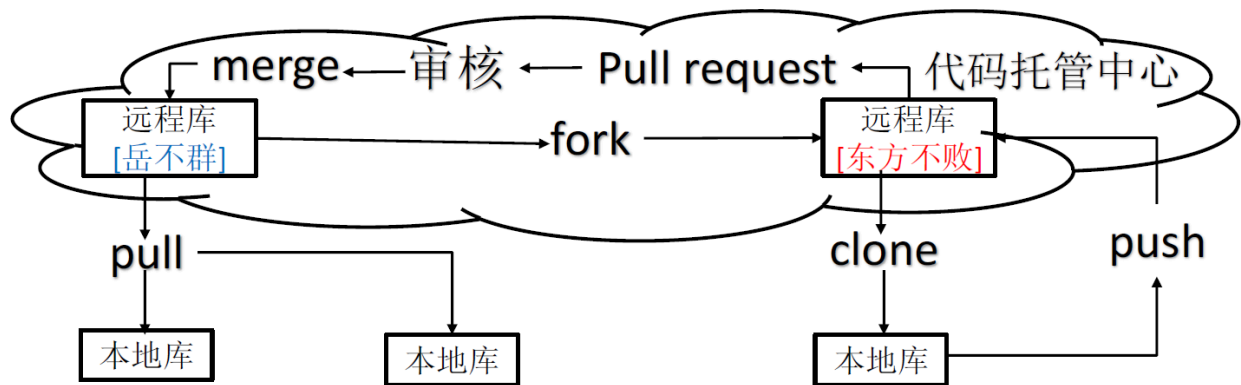




团队内合作



跨团队合作

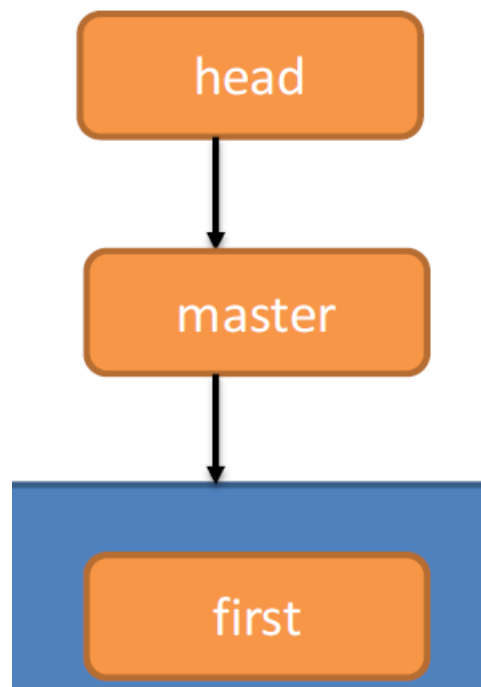


工作区：开发中对文件操作(增删改)后产生的区域，修改游离于分支外，git能够分辨出哪些文件被我们修改以及我们新增或者删除了哪些文件

暂存区可以说是一个缓存区，这个和内存映射相关，设计者之所以增加这个区域是为了提高查看修改前后diff的速度

目录 + git = git仓库

git切换版本 底层是移动的head指针



git command

```
// 初始化
git init

//目录下创建文件
//修改加入暂存区
git add .
//修改提交到本地仓库
git commit -m 'commit 001'
//查看日记
git log [option]
//--pretty=oneline 将提交信息显示为一行
//--abbrev-commit 使得输出的commitId更简短
//--graph 图显示

//精简方式显示提交记录
git-log
//找到倒数第2次提交的commitID
//版本问题
git reset commitID --hard

# // 修改加到暂存区
# // 输入以下命令，任务打包成一个stash节点，悬浮在开发分支上
git stash

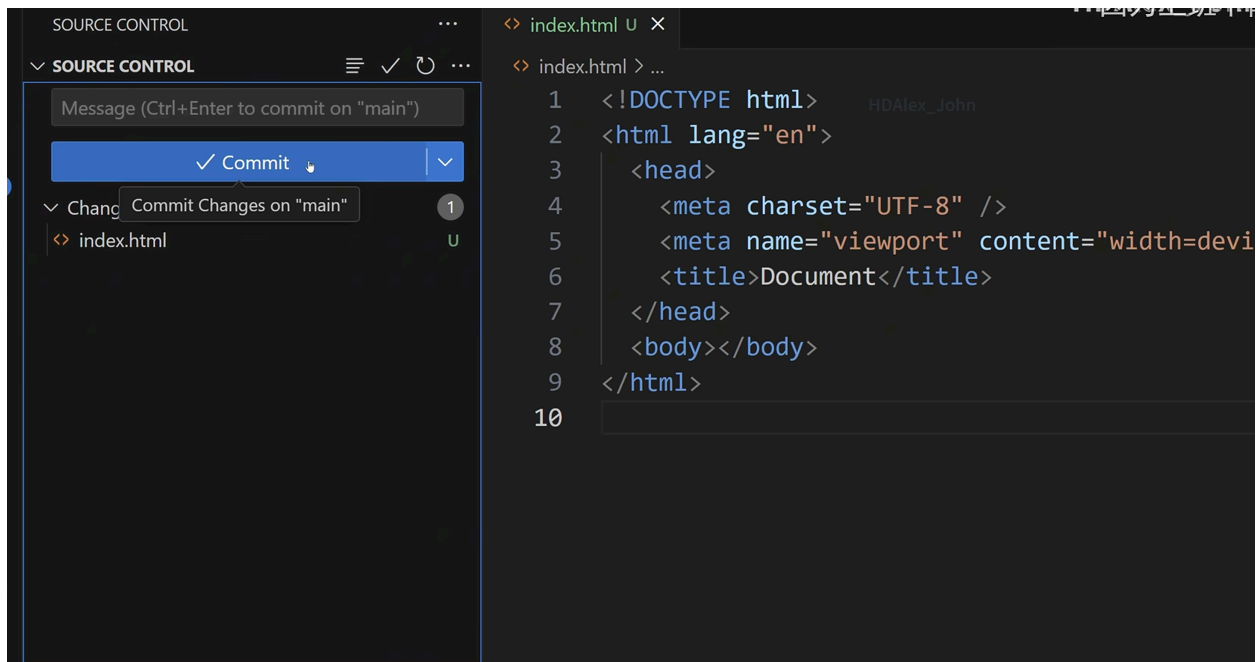
git stash pop: 弹出最新的一个保存，并删除暂存区中的记录
git stash apply : 恢复最新的一个保存，但并不删除暂存区中的记录
git stash drop stash{i}: 删除暂存区中的一个记录
git stash apply stash{i}: 恢复指定的保存记录

git reset --soft HEAD^ # 软回滚
```


`git reset --hard` # 结点哈希值 硬回滚 可以在同一条分支中的结点间自由切换

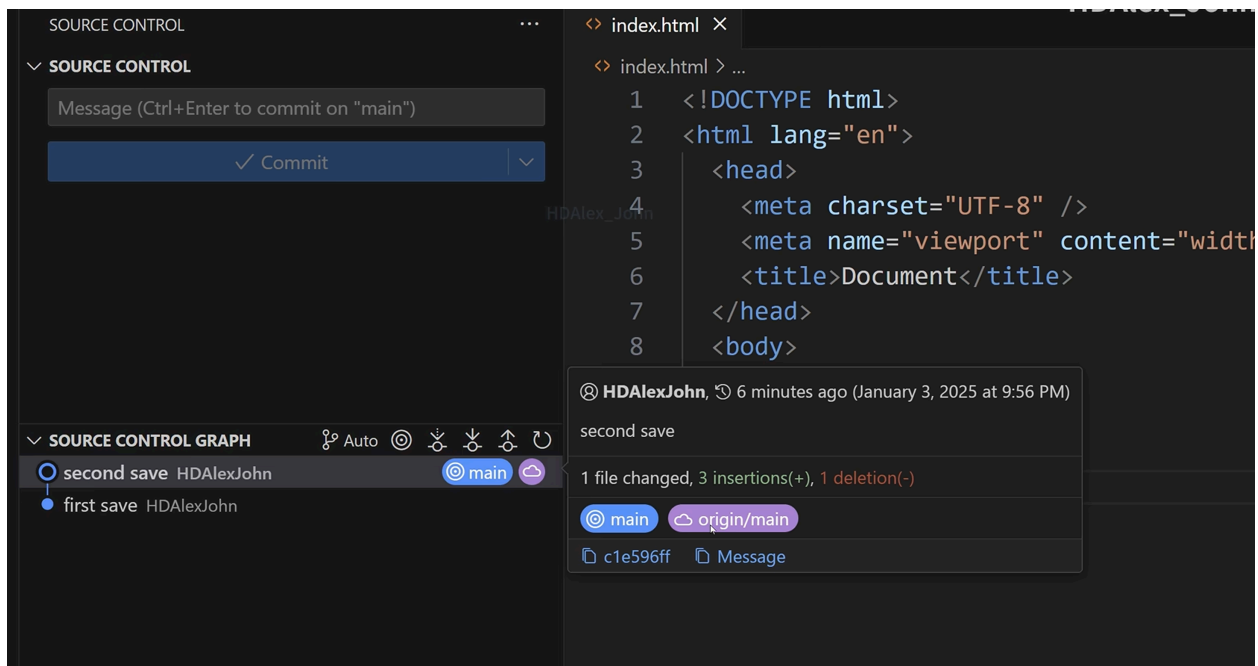
硬回滚后`git log` 看不到这个结点后面的提交记录，一个命令可以找回

`git reflog` # 罗列所有的操作结点，包括提交结点 可以看到已经删除的提交记录 查看把呢不能信息



一个文件夹里如果存在 `.git`，Git 会把它视为一个“独立的 Git 仓库”，不是普通文件夹。
嵌套仓库需要`git submodule add` 否则不是有效的submodule状态

`Remove-Item -Recurse -Force .git`



```

commit 58f6faab71404a6922e64987b76531149fa9da7f (HEAD -> main, origin/main)
Author: HDAlexJohn <fswwdza@gmail.com>
Date:   Fri Jan 3 22:03:48 2025 +0800

    add main.js

commit c1e596ffdcc763a3992d8f96bab2dd068f3033ab John
Author: HDAlexJohn <fswwdza@gmail.com>
Date:   Fri Jan 3 21:56:04 2025 +0800

    second save

commit 30555aef845794fec5f5dd2a1c345dce2fd5f8b4
Author: HDAlexJohn <fswwdza@gmail.com>
Date:   Fri Jan 3 21:54:52 2025 +0800

    first save
~

```

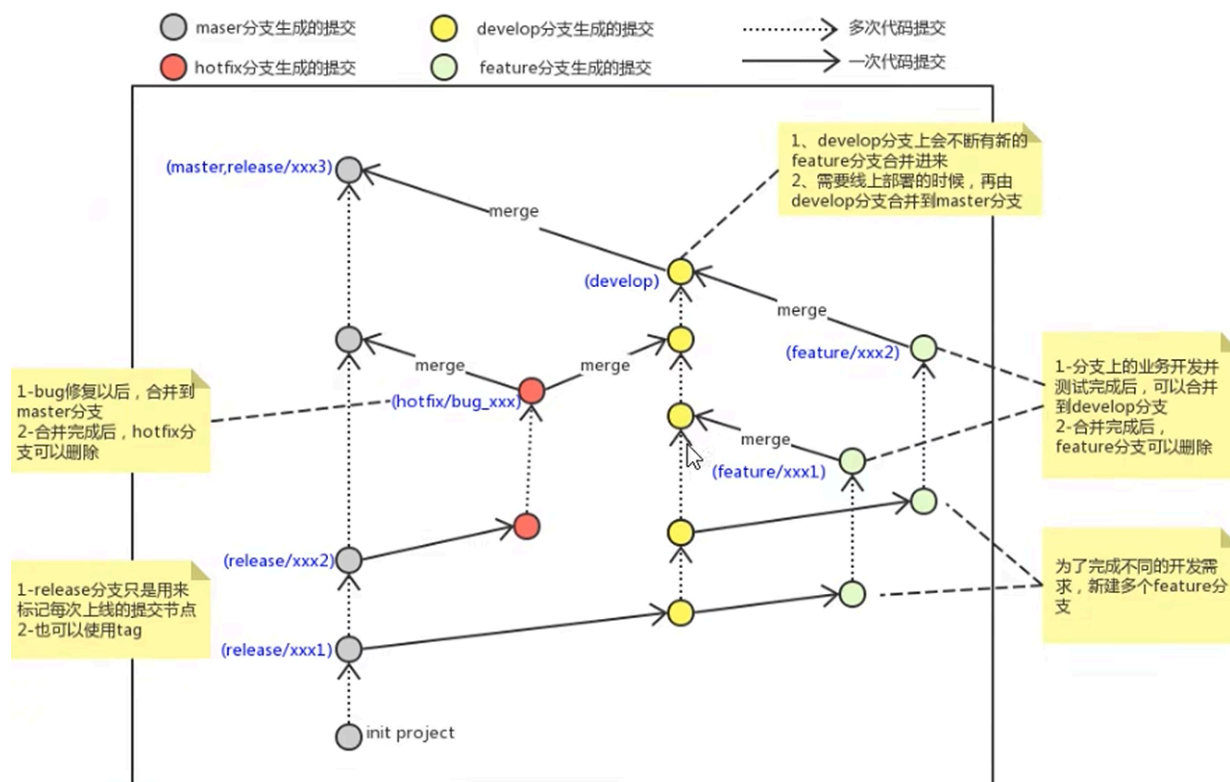
分支

develop：master创建分支主要开发功能 一般不删 其他修改提交到develop的分支之后可以删除

feature/xxx：从develop创建的分支，一般是同期并行开发，但不同期上线时创建的分支，分支上的研发任务完成后合并到develop分支

hotfix/xxxx：从master派生的分支，一般作为线上bug修复使用，修复完成后需要合并到master、test、develop分支

test, pre....



//建立本地分支和云端分支的关联，有的时候不清楚自己分支是否落后云端开发分支，建立一个关联是最简单的

//创建本地分支

```
git branch b1
```

```
git checkout -b b1 origin/云端分支名 // 基于远程分支建立本地分支
```

```
git checkout -b b1 //切换并创建不存在分支
```

```
git push --set-upstream origin 分支名，将本地分支推送到远端仓库并建立映射关系
```

```
git branch --unset-upstream //解除关联

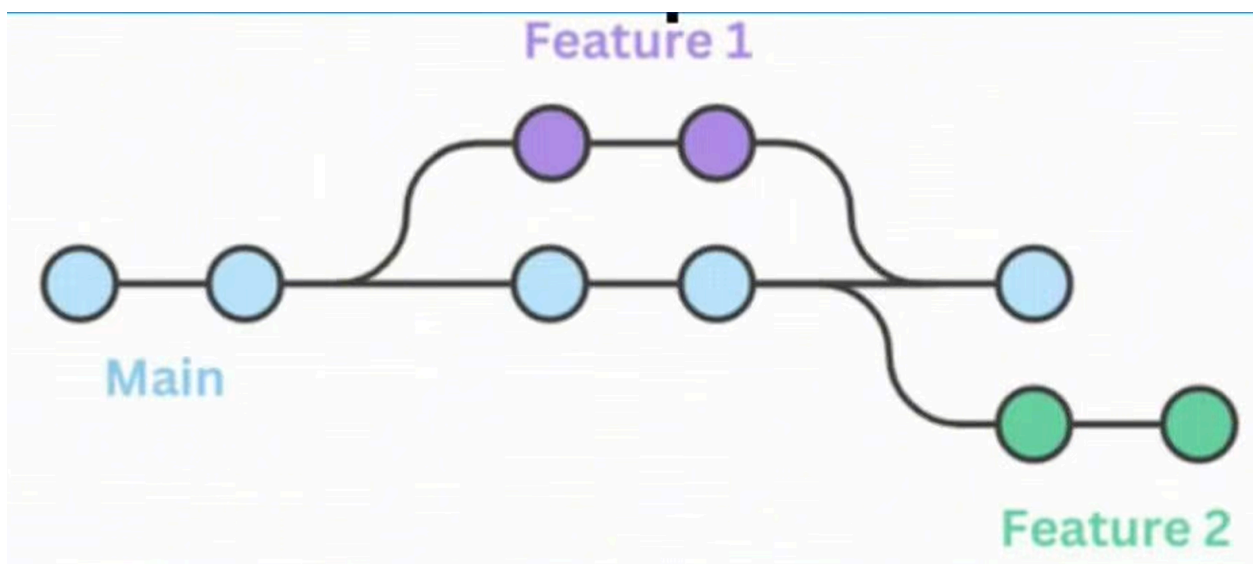
git merge b1 //一个分支提交合并到另一分支

// 删除本地分支 不能删除当前分支 只能删除其他分支
git branch -D b1 //不做任何删除 强制删除
git branch -d b1 //删除需要各种检查
// 删除云端分支
git push origin --delete 云端分支名

//创建file.txt
touch file.txt
```

回滚

merge



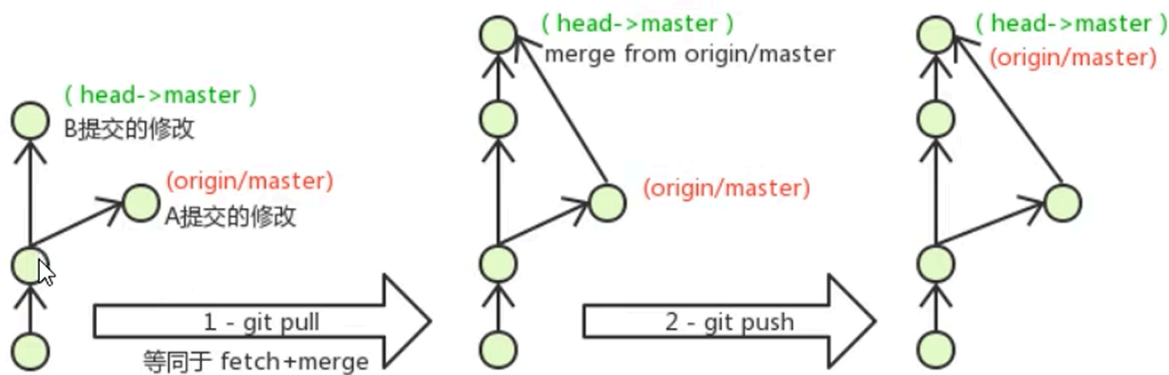
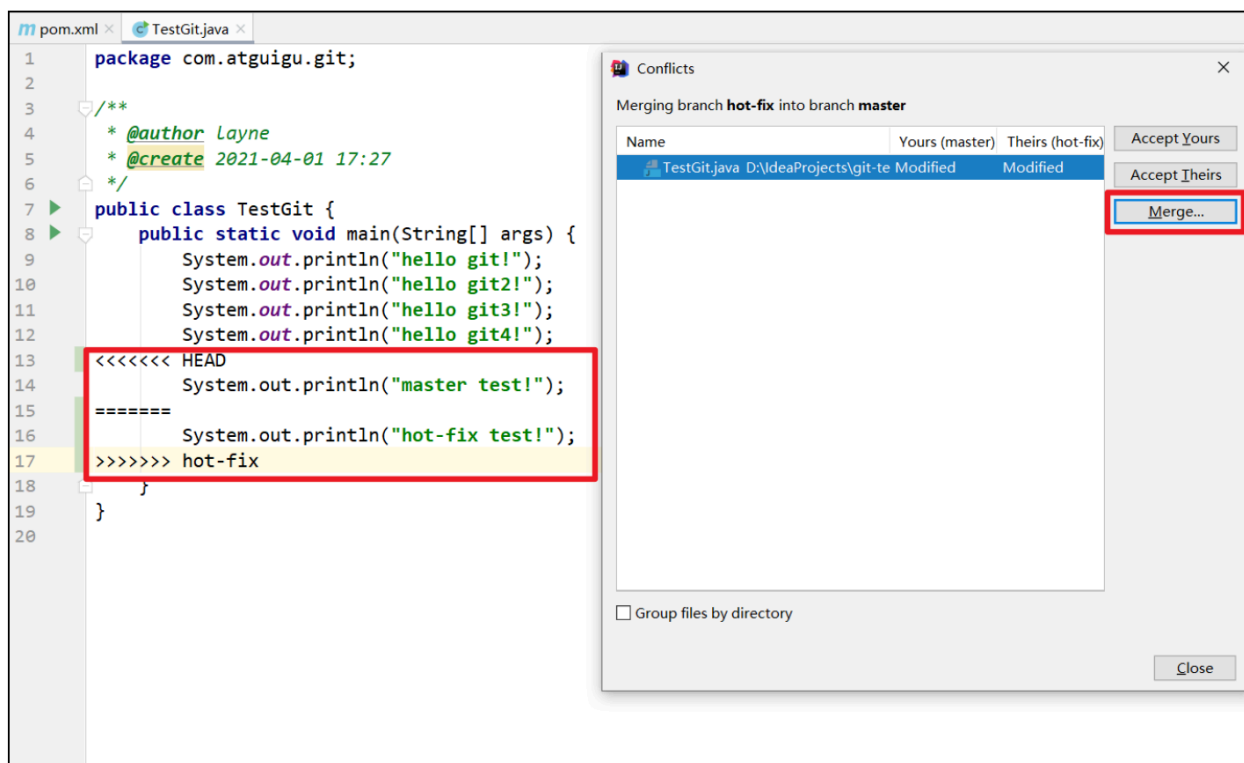
线上：main/master 运行稳定代码

测试环境：feature branch 新增代码基于master创建对应此次需求的feature分支开发

上线：feature→ master

conflict

双方动了同一块代码 双方都不知道对方动了



处理冲突地区 解决完冲突文件加入暂存区 commit

```

zhang@zhangmeng MINGW64 /c/czbk_advanced/git-test (master)
$ git merge dev
Auto-merging file01.txt
CONFLICT (content): Merge conflict in file01.txt 提示file01.txt文件中有冲突
Automatic merge failed; fix conflicts and then commit the result.

zhang@zhangmeng MINGW64 /c/czbk_advanced/git-test (master|MERGING)
$ cat file01.txt
<<<<<<< HEAD
count=1 <———— master分支上修改的内容
=====
count=2 <———— dev分支上修改的内容
>>>>>> dev

zhang@zhangmeng MINGW64 /c/czbk_advanced/git-test (master|MERGING)
$ vi file01.txt

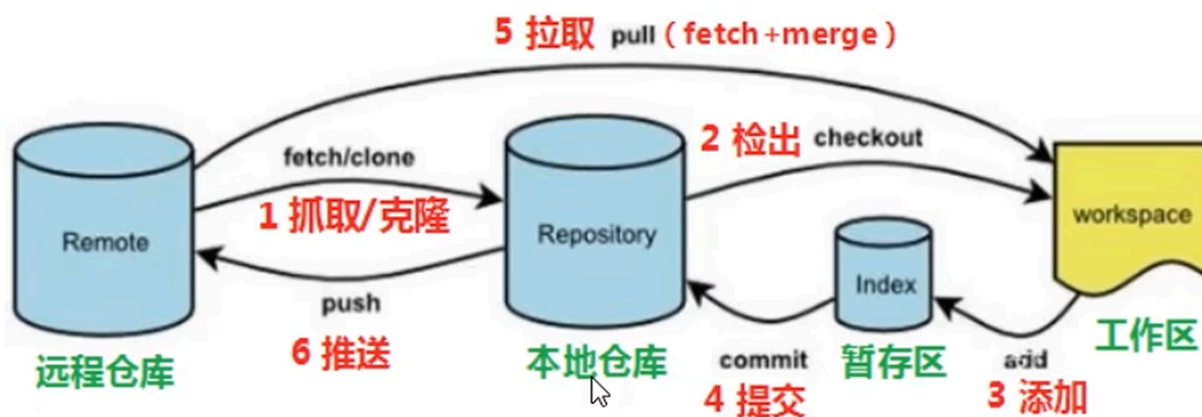
zhang@zhangmeng MINGW64 /c/czbk_advanced/git-test (master|MERGING)
$ cat file01.txt
count=2 <———— 修改成我们希望合并后的样子

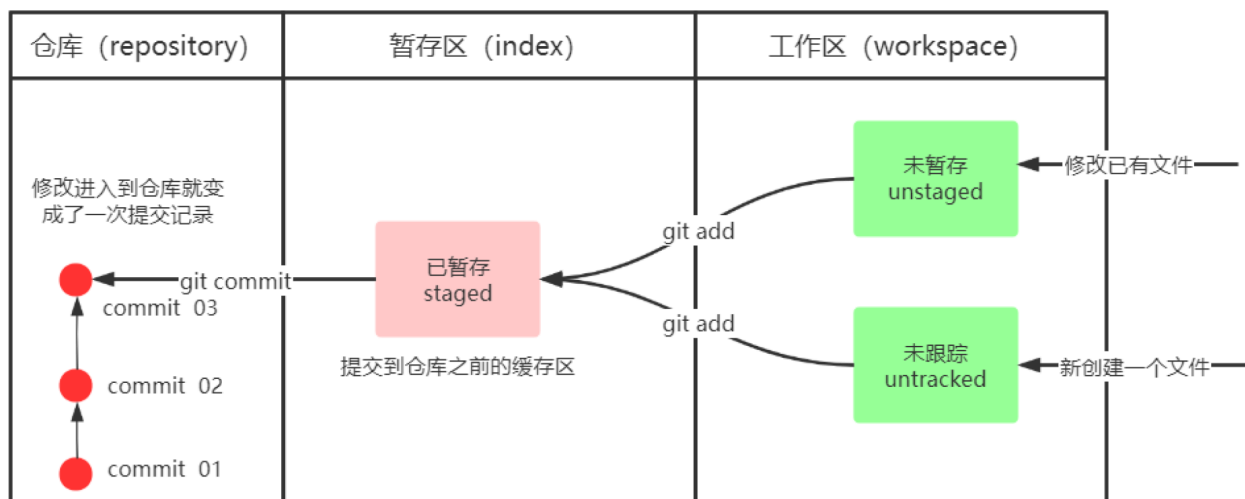
zhang@zhangmeng MINGW64 /c/czbk_advanced/git-test (master|MERGING)
$ git add . 添加到暂存区

zhang@zhangmeng MINGW64 /c/czbk_advanced/git-test (master|MERGING)
$ git commit 提交到本地仓库
[master 98eb214] Merge branch 'dev'

```

流程





配置SSH公钥

```
ssh-keygen -t rsa
```

获取公钥

```
cat ~/.ssh/id_rsa.pub
```

验证配置成功

```
ssh -T git@gitee.com
```

远程仓库默认SSH 而不是HTTPS

push成功 保证本地库版本要比远程库版本高

pull是拉去远端仓库代码到本地 如果云端库和本地库不一致 自动合并 如果合并失败需要手动改解决冲突

```
git remote add origin
```

```
// 推送分支到云端
```

```
//完整体 git push [-f][--set-upstream] [云端名称[本地分支][云端分支]] -f表示强制覆盖
```

```
git push origin b1:本地分支名
```

```
// 将你的本地分支推送合入到云端仓库的开发分支  
git push origin b1:云端开发分支名
```

```
git push --set-upstream origin master //推送到运算并建立和远端分支的关联关系  
//如果当前分支和云端分支关联 省略分支和云端名  
git push
```

```
// 如果你的本地分支有了新的修改，比如你回滚了，那这个时候你再推是不成功的  
git push origin b1:本地分支名 --force // 使用强制推送  
// 基本格式  
git push origin srcBranch:destBranch
```

```
git pull origin 远端分支名:b1 // 这里远端分支名和拉取后的本地分支名相同即可  
// 基本格式  
git pull origin srcBranch:destBranch
```

clone

从远程仓库抓取和拉取

```
git fetch[remote branch][local branch] //仓库更新抓取到本地不合并 如果不指定  
云端和分支 抓取所有分支
```

```
git pull[remote branch][local branch] //云端仓库修改拉到本地自动合并 fetch+m  
erge
```

```
git clone <path>[repository] //云端仓库clone到本地
```

git batch

git bash 命令行工具

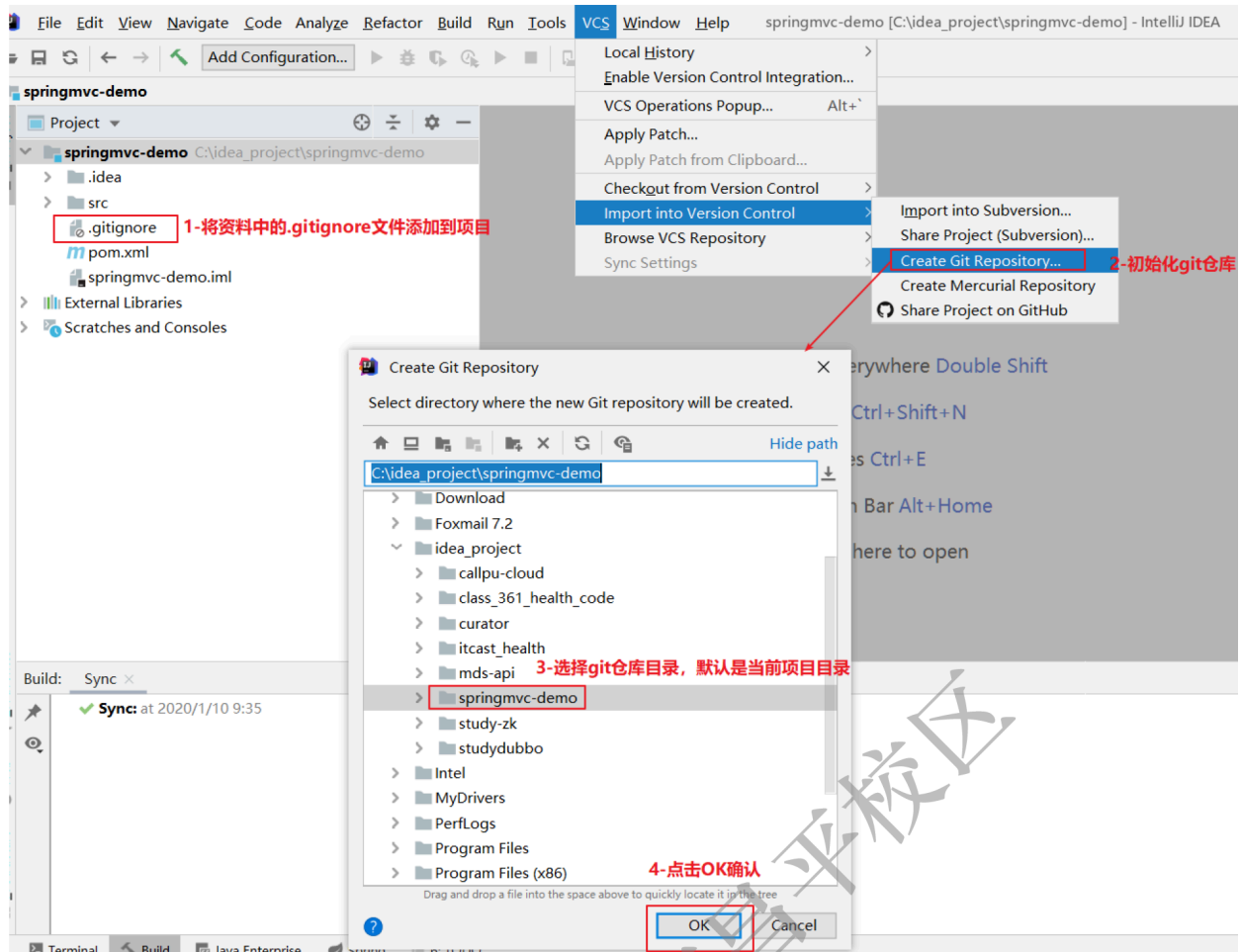
```
//解决乱码  
git config --global core.quotepath false  
  
//${git_home}/etc/bash.bashrc 文件最后加入下面两行  
export LANG="zh_CN.UTF-8"  
export LC_ALL="zh_CN.UTF-8"
```

git GUI 图形界面工具

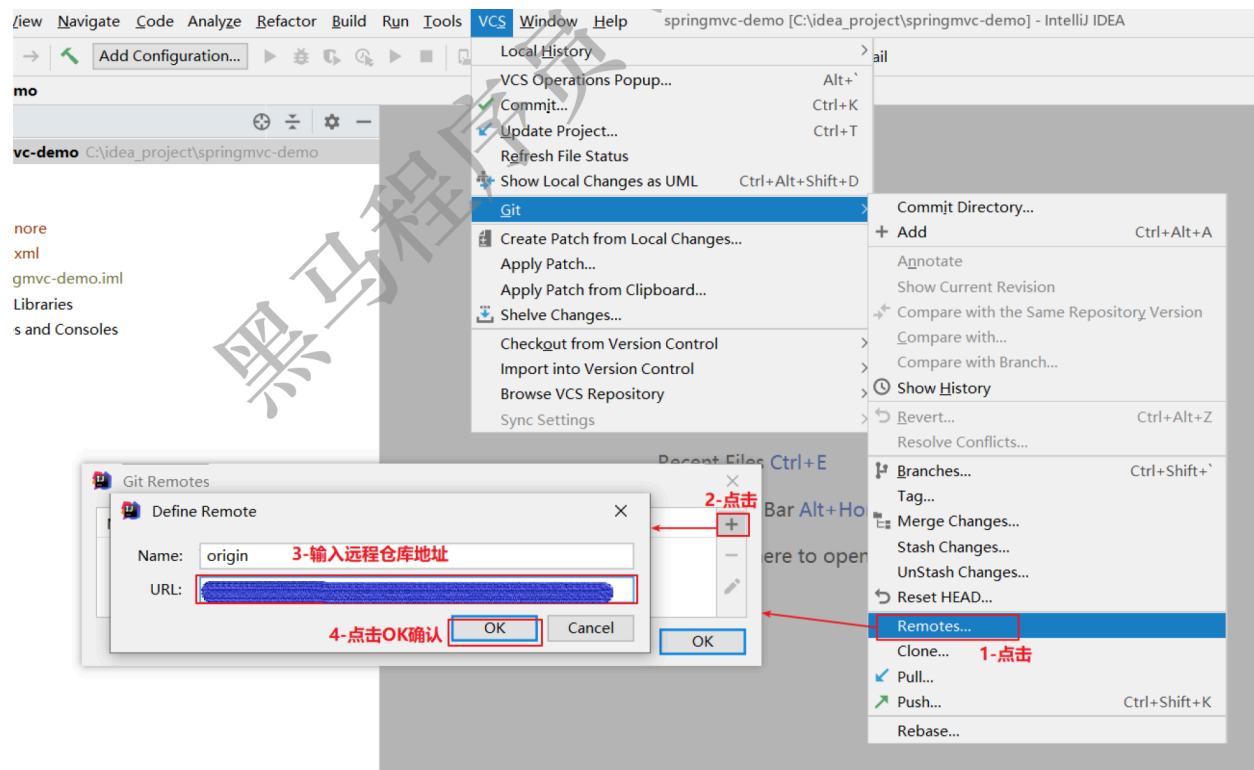
SVN

IDEA操作

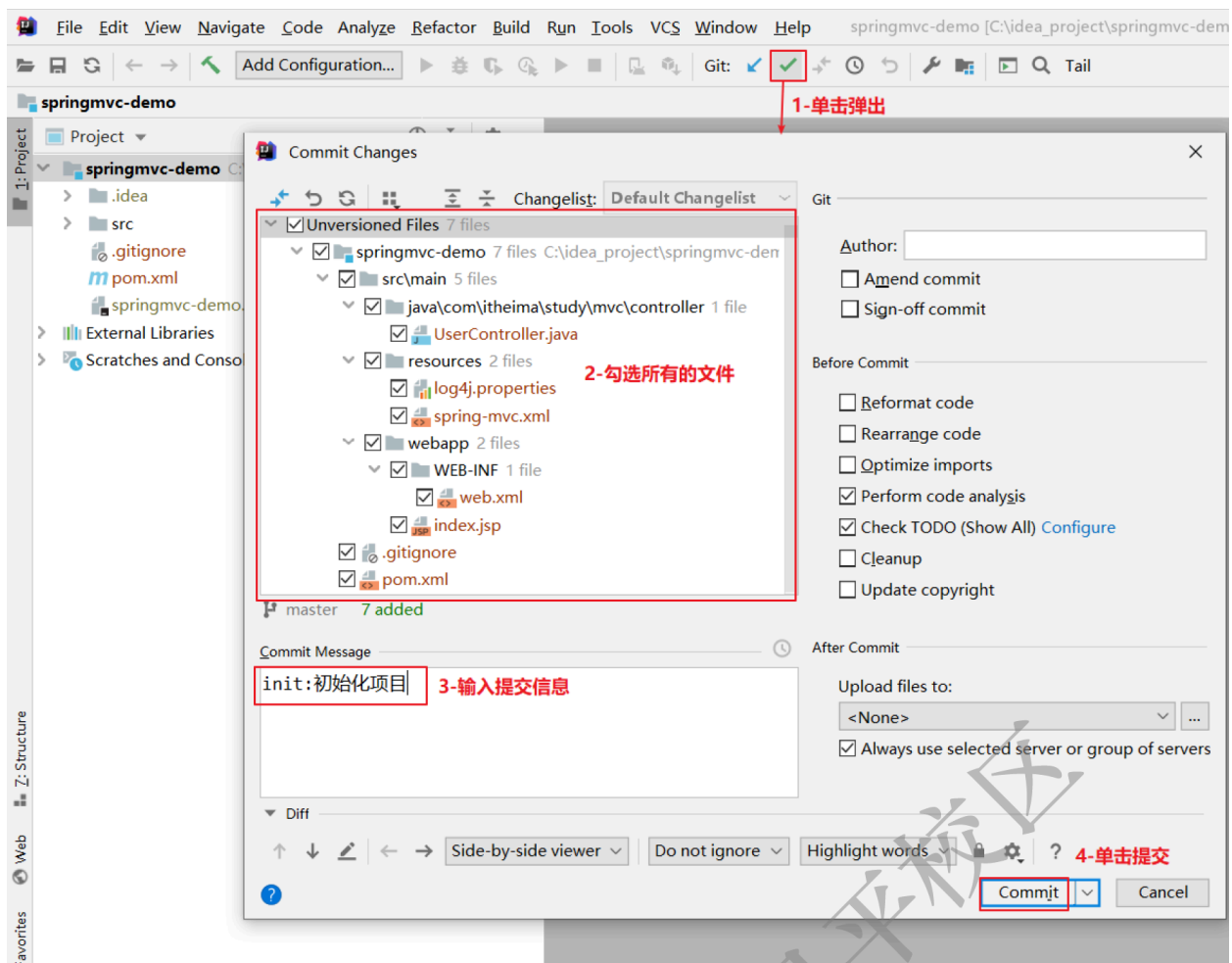
初始化本地仓库



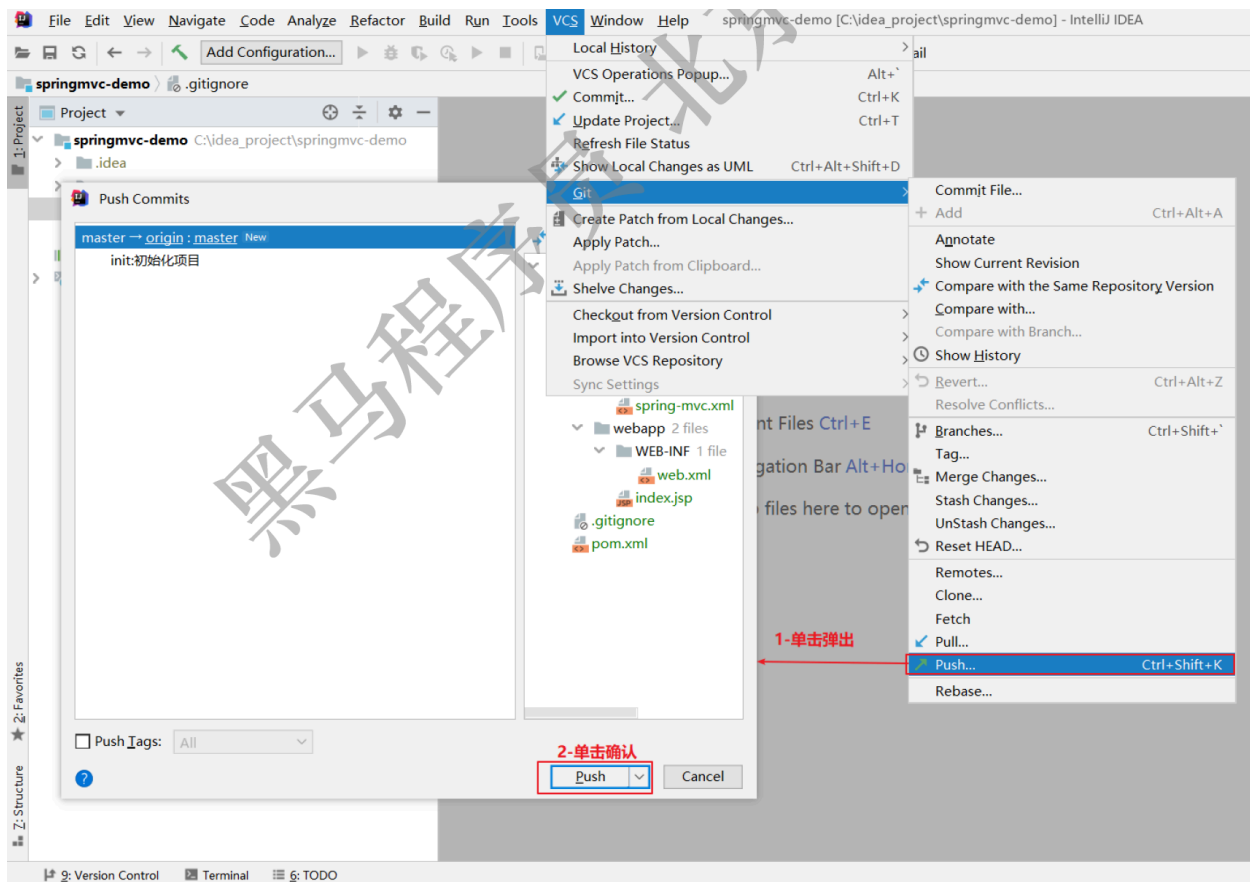
设置云端仓库



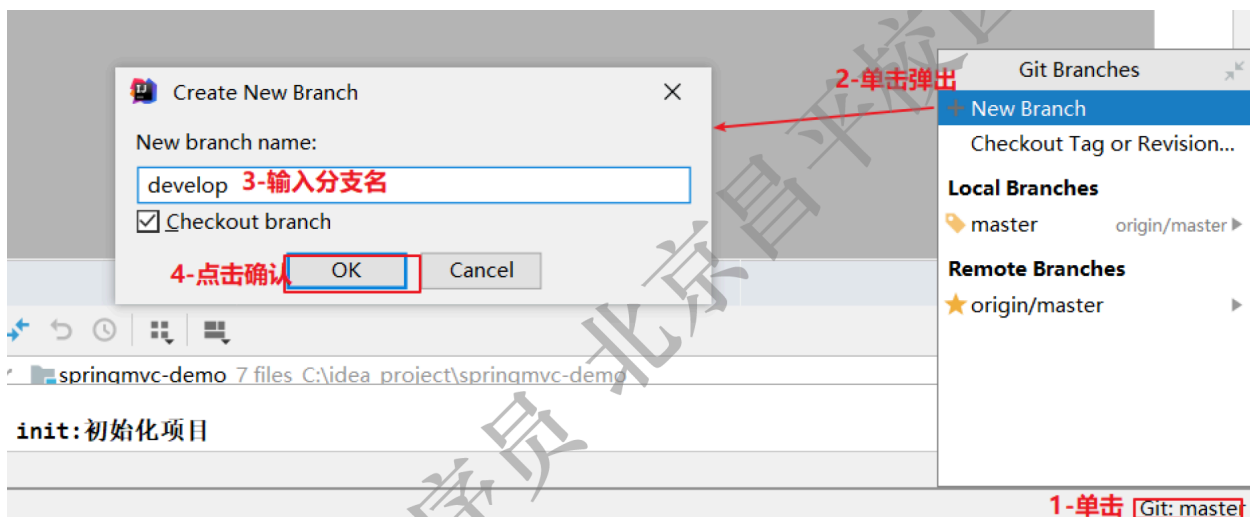
提交到本地仓库

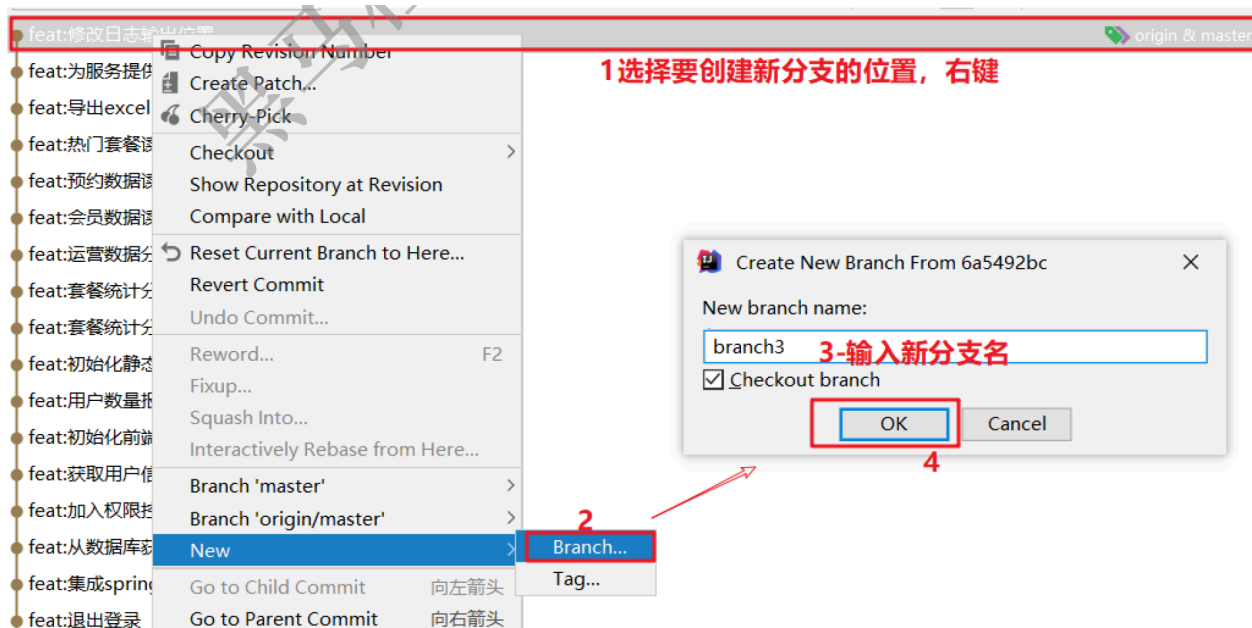


推送到云端仓库

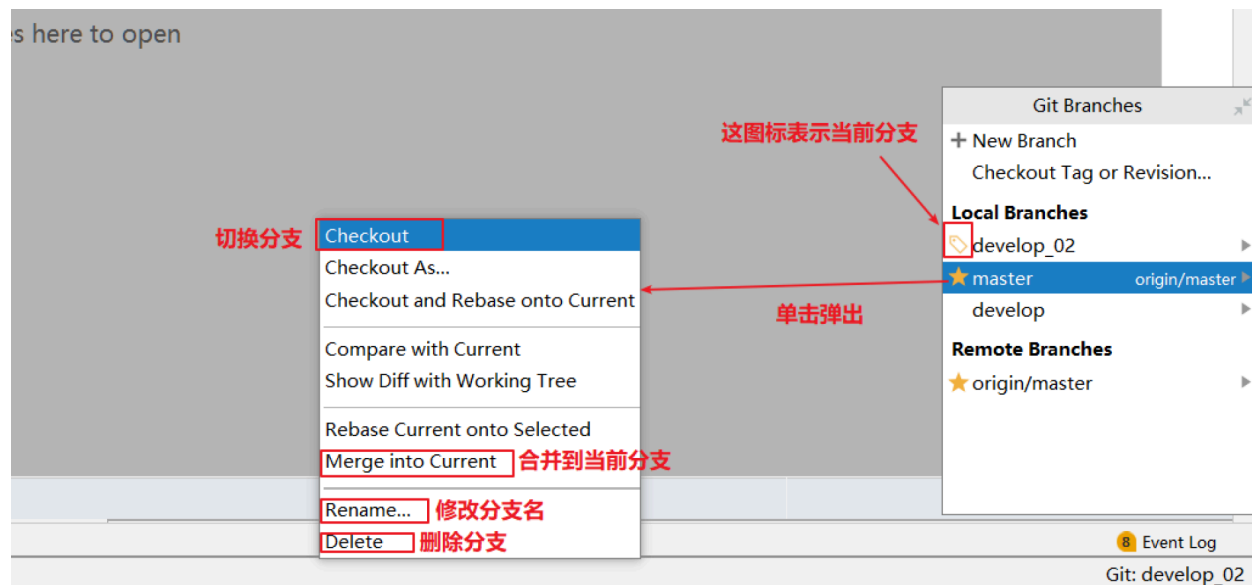


创建分支





切换分支



merge/pull可能发生冲突

